

# An Uncertainty-Aware Approach to Optimal Configuration of Stream Processing Systems

Pooyan Jamshidi

Imperial College London, UK  
Email: p.jamshidi@imperial.ac.uk

Giuliano Casale

Imperial College London, UK  
Email: g.casale@imperial.ac.uk

**Abstract**—Finding optimal configurations for Stream Processing Systems (SPS) is a challenging problem due to the large number of parameters that can influence their performance and the lack of analytical models to anticipate the effect of a change. To tackle this issue, we consider tuning methods where an experimenter is given a limited budget of experiments and needs to carefully allocate this budget to find optimal configurations. We propose in this setting Bayesian Optimization for Configuration Optimization (BO4CO), an auto-tuning algorithm that leverages Gaussian Processes (GPs) to iteratively capture posterior distributions of the configuration spaces and sequentially drive the experimentation. Validation based on Apache Storm demonstrates that our approach locates optimal configurations within a limited experimental budget, with an improvement of SPS performance typically of at least an order of magnitude compared to existing configuration algorithms.

## I. INTRODUCTION

We live in an increasingly instrumented world, where a large number of heterogeneous data sources typically provide continuous data streams from live stock markets, video sources, production line status feeds, and vital body signals [10]. Yet, the research literature lacks automated methods to support the configuration (*i.e.*, auto-tuning) of the underpinning SPSs. One possible explanation is that, “big data” systems such as SPSs often combine emerging technologies that are still poorly understood from a performance standpoint [14], [34] and therefore difficult to holistically configure. Hence there is a critical shortage of models and tools to anticipate the effects of changing a configuration in these systems. Examples of configuration parameters for a SPS include buffer size, heap sizes, serialization/de-serialization methods, among others.

Performance differences between a well-tuned configuration and a poorly configured one can be of orders of magnitude. Typically, administrators use a mix of rules-of-thumb, trial-and-error, and heuristic methods for setting configuration parameters. However, this way of testing and tuning is slow, and require skillful administrators with a good understanding of the SPS internals. Furthermore, decisions are also affected by the nonlinear interactions between configuration parameters.

In this paper, we address the problem of finding optimal configurations under these requirements: (i) a configuration space composed by multiple parameters; (ii) a limited budget of experiments that can be allocated to test the system; (iii) experimental results affected by uncertainties due to measurement inaccuracies or intrinsic variability in the system processing times. While the literature on auto-tuning work is abundant with existing solutions for databases, e-commerce and batch processing systems that address some of the above

challenges (*e.g.*, rule-based [16], design of experiment [29], model-based [19], [14], [34], [26], search-based [32], [22], [28], [8], [2], [33] and learning-based [3]), this is the first work to consider the problem under such constraints altogether.

In particular, we present a new auto-tuning algorithm called BO4CO that leverages GPs [31] to continuously estimate the mean and confidence interval of a response variable at yet-to-be-explored configurations. Using Bayesian optimization [25], the tuning process can account for all the available prior information about a system and the acquired configuration data, and apply a variety of kernel estimators [18] to locate regions where optimal configuration may lie. To the best of our knowledge, this is the first time that GPs are used for automated system configuration, thus a goal of the present work is to introduce and apply this class of machine learning methods into system performance tuning.

BO4CO is designed keeping in mind the limitations of sparse sampling from the configuration space. For example, its features include: (i) sequential planning to perform experiments that ensure coverage of the most promising zones; (ii) memorization of past-collected samples while planning new experiments; (iii) guarantees that optimal configurations will be eventually discovered by the algorithm. We show experimentally that BO4CO outperforms previous algorithms for configuration optimization. Our real configuration datasets are collected for three different SPS benchmark systems, implemented with Apache Storm, and using 5 cloud clusters worth several months of experimental time.

The rest of this paper is organized as follows. Section II discusses the motivations. The BO4CO algorithm is introduced in Section III and then validated in Section IV. Finally, Section V discusses the applicability of BO4CO in practice, Section VI reviews state of the art and Section VII concludes the paper.

## II. PROBLEM AND MOTIVATION

### A. Problem statement

In this paper, we focus on the problem of optimal system configuration defined as follows. Let  $X_i$  indicate the  $i$ -th configuration parameter, which takes values in a finite domain  $Dom(X_i)$ . In general,  $X_i$  may either indicate (i) integer variable such as “level of parallelism” or (ii) categorical variable such as “messaging frameworks” or Boolean parameter such as “enabling timeout”. Throughout the paper, by the term option, we mean possible values that can be assigned to a parameter. The *configuration space* is thus  $\mathbb{X} = Dom(X_1) \times \dots \times Dom(X_d)$ , which is the Cartesian product of the domains

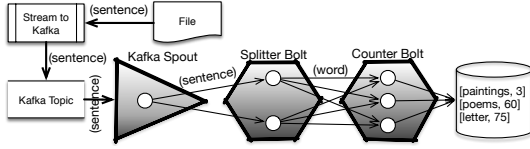


Fig. 1: WordCount topology architecture.

of  $d$  parameters of interest. We assume that each configuration  $\mathbf{x} \in \mathbb{X}$  is valid and denote by  $f(\mathbf{x})$  the response measured on the SPS under that configuration. Throughout, we assume that  $f$  is latency, however other response metrics (e.g., throughput) may be used. The graph of  $f$  over configurations is called the *response surface* and it is partially observable, i.e., the actual value of  $f(\mathbf{x})$  is known only at points  $\mathbf{x}$  that has been previously experimented with. We here consider the problem of finding an optimal configuration  $\mathbf{x}^*$  that minimizes  $f$  over the configuration space  $\mathbb{X}$  with as few experiments as possible:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathbb{X}} f(\mathbf{x}) \quad (1)$$

In fact, the response function  $f(\cdot)$  is usually unknown or partially known, i.e.,  $y_i = f(\mathbf{x}_i)$ ,  $\mathbf{x}_i \in \mathbb{X}$ . In practice, such measurements may contain noise, i.e.,  $y_i = f(\mathbf{x}_i) + \epsilon_i$ . Note that since the response surface is only partially-known, finding the optimal configuration is a blackbox optimization problem [18], [24], which is also subject to noise. In fact, the problem of finding an optimal solution of a non-convex and multi-modal response surface (cf. Figure 2) is  $\mathcal{NP}$ -hard [30]. Therefore, on instances where it is impossible to locate a global optimum, BO4CO will strive to find the best possible local optimum within the available experimental budget.

### B. Motivation

1) *A running example:* WordCount (cf. Figure 1) is a popular benchmark SPS. In WordCount a text file is fed to the system and it counts the number of occurrences of the words in the text file. In Storm, this corresponds to the following operations. A Processing Element (PE) called Spout is responsible to read the input messages (tuples) from a data source (e.g., a Kafka topic) and stream the messages (i.e., sentences) to the topology. Another PE of type Bolt called Splitter is responsible for splitting sentences into words, which are then counted by another PE called Counter.

2) *Nonlinear interactions:* We now illustrate one of the inherent challenges of configuration optimization. The metric that defines the surface in Figure 2 is the *latency* of individual messages, defined as the time since emission by the Kafka Spout to completion at the Counter, see Figure 1. Note that this function is the subset of  $\text{wc}(6D)$  in Table I when the level of parallelism of Splitters and Counters is varied in [1, 6] and [1, 18]. The surface is strongly *non-linear* and *multi-modal* and indicates two important facts. First, the performance difference between the best and worst settings is substantial, 65%, and with more intense workloads we have observed differences in latency as large as 99%, see [1]. Next, non-linear relations between the parameters imply that the optimal number of counters depends on the number of Splitters, and vice-versa. Figure 3 shows this *non-linear interaction* [26] and

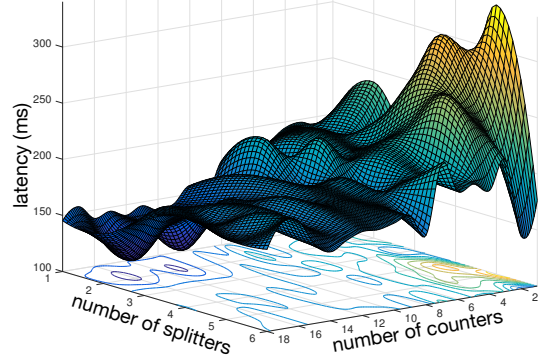


Fig. 2: WordCount response surface. It is an interpolated surface and is a projection of 6 dimensions, in  $\text{wc}(6D)$ , onto 2D. It shows the non-convexity, multi-modality and the substantial performance difference between different configurations.

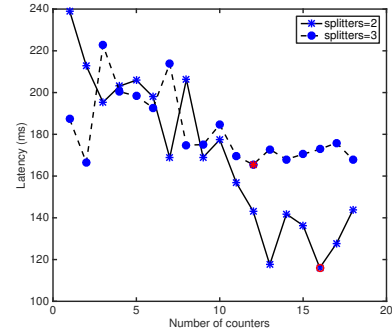


Fig. 3: WordCount latency, cut through Figure 2.

demonstrates that if one tries to minimize latency by acting just on one of these parameters at the time, the resulting configuration may not lead to a global optimum, as the number of Splitters has a strong influence on the optimal counters.

3) *Sparsity of effects:* Another observation from our extensive experiments with SPS is the *sparsity of effects*. More specifically, this means low-order interactions among a few dominating factors can explain the main changes in the response function observed in the experiments. In this work we assume sparsity of effects, which also helps in addressing the intractable growth of the configuration space [15].

*Methodology.* In order to verify to what degree the sparsity of effects assumption holds in SPS, we ran experiments on 3 different benchmarks that exhibit different bottlenecks: WordCount ( $\text{wc}$ ) is CPU intensive, RollingSort ( $\text{rs}$ ) is memory intensive, and SOL ( $\text{sol}$ ) is network intensive. Different testbed settings were also considered, for a total of 5 datasets, as listed in Table I. Note that the parameters we consider here are known to significantly influence latency, as they have been chosen according to professional tuning guides [21] and also small scale tests where we varied a single parameter to make sure that the selected parameters were all influential. For each test in the experiment, we run the benchmark for 8 minutes including the initial burn-in period. Further details on the experimental procedure are given in Section IV-B. Note that the largest dataset (i.e.,  $\text{rs}(6D)$ ) has required alone  $3840 \times 8/60/24 = 21$  days, within a total experimental time of about 2.5 months to collect the datasets of Table I.

TABLE I: Sparsity of effects on 5 experiments where we have varied different subsets of parameters and used different testbeds. Note that these are the datasets we experimentally measured on the benchmark systems and we use them for the evaluation, more details including the results for 6 more experiments are in the online appendix.

|   | Topol.  | Parameters                                                                     | Main factors | Merit | Size | Testbed |
|---|---------|--------------------------------------------------------------------------------|--------------|-------|------|---------|
| 1 | wc(6D)  | 1-spouts, 2-max_spout, 3-spout_wait, 4-splitters, 5-counters, 6-netty_min_wait | {1, 2, 5}    | 0.787 | 2880 | C1      |
| 2 | sol(6D) | 1-spouts, 2-max_spout, 3-top_level, 4-netty_min_wait, 5-message_size, 6-bolts  | {1, 2, 3}    | 0.447 | 2866 | C2      |
| 3 | rs(6D)  | 1-spouts, 2-max_spout, 3-sorters, 4-emit_freq, 5-chunk_size, 6-message_size    | {3}          | 0.385 | 3840 | C3      |
| 4 | wc(3D)  | 1-max_spout, 2-splitters, 3-counters                                           | {1, 2}       | 0.480 | 756  | C4      |
| 5 | wc(5D)  | 1-spouts, 2-splitters, 3-counters, 4-buffer-size, 5-heap                       | {1}          | 0.851 | 1080 | C5      |

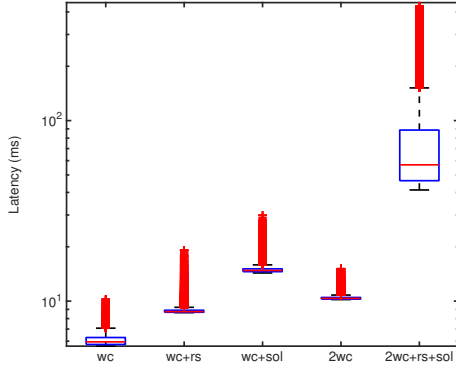


Fig. 4: Noisy experimental measurements. Note that + here means that wc is deployed in a multi-tenant environment with other topologies and as a result not only the latency is increased but also the variability became greater.

*Results.* After collecting experimental data, we have used a common correlation-based feature selector<sup>1</sup> implemented in Weka to rank parameter subsets according to a heuristic. The bias of the merit function is toward subsets that contain parameters that are highly correlated with the response variable. Less influential parameters are filtered because they will have low correlation with latency, and a set with the main factors is returned. For all of the 5 datasets, we list in Table I the main factors. The analysis results demonstrate that in all the 5 experiments at most 2-3 parameters were strongly interacting with each other, out of a maximum of 6 parameters varied simultaneously. Therefore, the determination of the regions where performance is optimal will likely be controlled by such dominant factors, even though the determination of a global optimum will still depends on all the parameters.

4) *Measurement uncertainty:* We now illustrate measurement variabilities, which represent an additional challenge for configuration optimization. As depicted in Figure 4, we took

<sup>1</sup>The most significant parameters are selected based on the following merit function [7], also shown in Table I:

$$m_{ps} = \frac{n\overline{r_{lp}}}{\sqrt{n + n(n-1)\overline{r_{pp}}}}, \quad (2)$$

where  $\overline{r_{lp}}$  is the mean parameter-latency correlation,  $n$  is the number of parameters,  $\overline{r_{pp}}$  is the average feature-feature inter-correlation [7, Sec 4.4].

different samples of the latency metric over 2 hours for five different deployments of WordCount. The experiments run on a multi-node cluster on the EC2 cloud. After filtering the initial burn-in, we computed averages and standard deviation of the latencies. Note that the configuration across all 5 settings is similar, the only difference is the number of co-located topologies in the testbed. The data in boxplots illustrate that variability can be small in some settings (e.g., wc), while they can be large in some other experimental setups (e.g., 2wc+rs+sol). In traditional techniques such as design of experiments, such variability is addressed by repeating experiments multiple times and obtaining regression estimates for the system model across such repetitions. However, we here pursue the alternative approach of relying on GP models to capture both mean and variance of measurements within the model that guides the configuration process. The theory underpinning this approach is discussed in the next section.

### III. BO4CO: BAYESIAN OPTIMIZATION FOR CONFIGURATION OPTIMIZATION

#### A. Bayesian Optimization with Gaussian Process prior

Bayesian optimization is a sequential design strategy that allows us to perform global optimization of blackbox functions [25]. The main idea of this method is to treat the blackbox objective function  $f(\mathbf{x})$  as a random variable with a given prior distribution, and then perform optimization on the posterior distribution of  $f(\mathbf{x})$  given experimental data. In this work, GPs are used to model this blackbox objective function at each point  $\mathbf{x} \in \mathbb{X}$ . That is, let  $\mathbb{S}_{1:t}$  be the experimental data collected in the first  $t$  iterations and let  $\mathbf{x}_{t+1}$  be a candidate configuration that we may select to run the next experiment. Then BO4CO assesses the probability that this new experiment could find an optimal configuration using the posterior distribution:

$$\Pr(f_{t+1}|\mathbb{S}_{1:t}, \mathbf{x}_{t+1}) \sim \mathcal{N}(\mu_t(\mathbf{x}_{t+1}), \sigma_t^2(\mathbf{x}_{t+1})),$$

where  $\mu_t(\mathbf{x}_{t+1})$  and  $\sigma_t^2(\mathbf{x}_{t+1})$  are suitable estimators of the mean and standard deviation of a normal distribution that is used to model this posterior. The main motivation behind the choice of GPs as prior here is that it offers a framework in which reasoning can be not only based on mean estimates but also the variance, providing more informative decision makings. The other reason is that all the computations in this framework are based on *linear algebra*.

Figure 5 illustrates the GP-based Bayesian optimization using a 1-dimensional response surface. The curve in blue is the unknown true posterior distribution, whereas the mean is shown in green and the 95% confidence interval at each point in the shaded area. Stars indicate measurements carried out in the past and recorded in  $\mathbb{S}_{1:t}$  (i.e., observations). Configuration corresponds to  $\mathbf{x}_1$  has a large confidence interval due to lack of observations in its neighborhood. Conversely,  $\mathbf{x}_4$  has a narrow confidence interval since neighboring configurations have been experimented with. The confidence interval in the neighborhood of  $\mathbf{x}_2$  and  $\mathbf{x}_3$  is not high and correctly our approach does not decide to explore these zones. The next configuration  $\mathbf{x}_{t+1}$ , indicated by a small circle right to the  $\mathbf{x}_4$ , is selected based on a criterion that will be defined later.

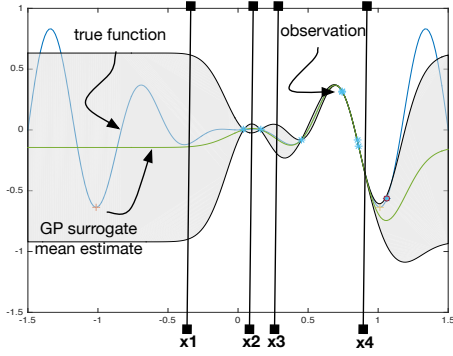


Fig. 5: An example of 1D GP model: GPs provide mean estimates as well as the uncertainty in estimations, i.e., variance.

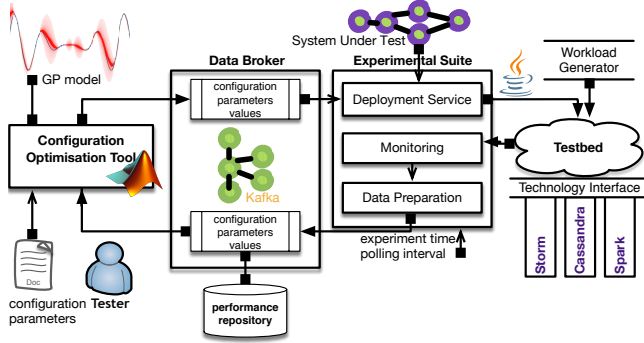


Fig. 6: BO4CO architecture: (i) optimization and (ii) experimental suite are integrated via (iii) a data broker. The integrated solution is available: <https://github.com/dice-project/DICE-Configuration-BO4CO>.

### B. BO4CO algorithm

BO4CO's high-level architecture is shown in Figure 6 and the procedure that drives the optimization is described in Algorithm 1. We start by bootstrapping the optimization following Latin Hypercube Design (lhd) to produce an initial design  $\mathcal{D} = \{x_1, \dots, x_n\}$  (cf. *step 1* in Algorithm 1). Although other design approaches (e.g., random) could be used, we have chosen lhd because: (i) it ensures that the configuration samples in  $\mathcal{D}$  is representative of the configuration space  $\mathbb{X}$ , whereas traditional random sampling [17], [9] (called brute-force) does not guarantee this [20]; (ii) another advantage is that the lhd samples can be taken one at a time, making it efficient in high dimensional spaces. After obtaining the measurements regarding the initial design, BO4CO then fits a GP model to the design points  $\mathcal{D}$  to form our belief about the underlying response function (cf. *step 3* in Algorithm 1). The while loop in Algorithm 1 iteratively updates the belief until the budget runs out: As we accumulate the data  $\mathbb{S}_{1:t} = \{(x_i, y_i)\}_{i=1}^t$ , where  $y_i = f(x_i) + \epsilon_i$  with  $\epsilon \sim \mathcal{N}(0, \sigma^2)$ , a prior distribution  $\Pr(f)$  and the likelihood function  $\Pr(\mathbb{S}_{1:t}|f)$  form the posterior distribution:  $\Pr(f|\mathbb{S}_{1:t}) \propto \Pr(\mathbb{S}_{1:t}|f) \Pr(f)$ .

A GP is a distribution over functions [31], specified by its mean (see Section III-E2), and covariance (see Section III-E1):

$$y = f(x) \sim \mathcal{GP}(\mu(x), k(x, x')), \quad (3)$$

### Algorithm 1 : BO4CO

**Input:** Configuration space  $\mathbb{X}$ , Maximum budget  $N_{max}$ , Response function  $f$ , Kernel function  $K_\theta$ , Hyper-parameters  $\theta$ , Design sample size  $n$ , learning cycle  $N_l$

**Output:** Optimal configurations  $x^*$  and learned model  $\mathcal{M}$

- 1: choose an initial sparse design (lhd) to find an initial design samples  $\mathcal{D} = \{x_1, \dots, x_n\}$
- 2: obtain *performance measurements* of the initial design,  $y_i \leftarrow f(x_i) + \epsilon_i, \forall x_i \in \mathcal{D}$
- 3:  $\mathbb{S}_{1:n} \leftarrow \{(x_i, y_i)\}_{i=1}^n; t \leftarrow n + 1$
- 4:  $\mathcal{M}(x|\mathbb{S}_{1:n}, \theta) \leftarrow$  fit a GP model to the design  $\triangleright$  Eq.(3)
- 5: **while**  $t \leq N_{max}$  **do**
- 6:   if  $(t \bmod N_l = 0)$   $\theta \leftarrow$  learn the kernel hyper-parameters by maximizing the likelihood
- 7:   find *next configuration*  $x_t$  by optimizing the selection criteria over the estimated response surface given the data,  $x_t \leftarrow \arg \max_x u(x|\mathcal{M}, \mathbb{S}_{1:t-1})$   $\triangleright$  Eq.(9)
- 8:   obtain performance for the *new configuration*  $x_t, y_t \leftarrow f(x_t) + \epsilon_t$
- 9:   Augment the configuration  $\mathbb{S}_{1:t} = \{\mathbb{S}_{1:t-1}, (x_t, y_t)\}$
- 10:    $\mathcal{M}(x|\mathbb{S}_{1:t}, \theta) \leftarrow$  re-fit a new GP model  $\triangleright$  Eq.(7)
- 11:    $t \leftarrow t + 1$
- 12: **end while**
- 13:  $(x^*, y^*) = \min_{\mathbb{S}_{1:N_{max}}}$
- 14:  $\mathcal{M}(x)$

where  $k(x, x')$  defines the distance between  $x$  and  $x'$ . Let us assume  $\mathbb{S}_{1:t} = \{(x_{1:t}, y_{1:t}) | y_i := f(x_i)\}$  be the collection of  $t$  observations. The function values are drawn from a multi-variate Gaussian distribution  $\mathcal{N}(\mu, K)$ , where  $\mu := \mu(x_{1:t})$ ,

$$K := \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_t) \\ \vdots & \ddots & \vdots \\ k(x_t, x_1) & \dots & k(x_t, x_t) \end{bmatrix} \quad (4)$$

In the while loop in BO4CO, given the observations we accumulated so far, we intend to fit a new GP model:

$$\begin{bmatrix} f_{1:t} \\ f_{t+1} \end{bmatrix} \sim \mathcal{N}(\mu, \begin{bmatrix} K + \sigma^2 I & k \\ k^\top & k(x_{t+1}, x_{t+1}) \end{bmatrix}), \quad (5)$$

where  $k(x)^\top = [k(x, x_1) \ k(x, x_2) \ \dots \ k(x, x_t)]$  and  $I$  is identity matrix. Given the Eq. (5), the new GP model can be drawn from this new Gaussian distribution:

$$\Pr(f_{t+1}|\mathbb{S}_{1:t}, x_{t+1}) = \mathcal{N}(\mu_t(x_{t+1}), \sigma_t^2(x_{t+1})), \quad (6)$$

where

$$\mu_t(x) = \mu(x) + k(x)^\top (K + \sigma^2 I)^{-1} (y - \mu) \quad (7)$$

$$\sigma_t^2(x) = k(x, x) + \sigma^2 I - k(x)^\top (K + \sigma^2 I)^{-1} k(x) \quad (8)$$

These posterior functions are used to select the next point  $x_{t+1}$  as detailed in Section III-C.

### C. Configuration selection criteria

The selection criteria is defined as  $u : \mathbb{X} \rightarrow \mathbb{R}$  that selects  $x_{t+1} \in \mathbb{X}$ , should  $f(\cdot)$  be evaluated next (*step 7*):

$$x_{t+1} = \arg \max_{x \in \mathbb{X}} u(x|\mathcal{M}, \mathbb{S}_{1:t}) \quad (9)$$



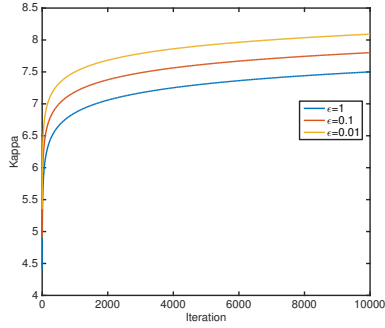


Fig. 7: Change of  $\kappa$  value over time: it begins with a small value to exploit the mean estimates and it increases over time in order to explore.

Although several different criteria exist in the literature (see [25]), BO4CO uses *Lower Confidence Bound* (LCB) [25] based on empirical results (see [1]). LCB selects the next configuration by balancing exploitation and exploration:

$$u_{LCB}(\mathbf{x}|\mathcal{M}, \mathbb{S}_{1:n}) = \underset{\mathbf{x} \in \mathbb{X}}{\operatorname{argmin}} \mu_t(\mathbf{x}) - \kappa \sigma_t(\mathbf{x}), \quad (10)$$

where  $\kappa$  can be set according to the objectives. For instance, if we require to find a near optimal configuration quickly we set a low value to  $\kappa$  to take the most out of the initial design knowledge. However, if we want to skip local minima, we can set a high value to  $\kappa$ . Furthermore,  $\kappa$  can be adapted over time to benefit from the both [13]. For instance,  $\kappa$  can start with a reasonably small value to exploit the initial design and increase over time to do more explorations (cf. Figure 7).

#### D. Illustration

The steps in Algorithm 1 are illustrated in Figure 8. Firstly, an initial design based on lhd is produced (Figure 8(a)). Secondly, a GP model is fit to the initial design (Figure 8(b)). Then, the model is used to calculate the selection criteria (Figure 8(c)). Finally, the configuration that maximizes the selection criteria is used to run the next experiment and provide data for refitting a more accurate model (Figure 8(d)).

#### E. Model fitting in BO4CO

In this section, we provide some practical considerations to make GPs applicable for configuration optimization.

1) *Kernel function*: In BO4CO, as shown in Algorithm 1, the covariance function  $k : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{R}$  dictates the structure of the response function we fit to the observed data. For integer variables (cf. Section II-A), we implemented the Matérn kernel [31]. The main reason behind this choice is that along each dimension in the configuration response functions different level of smoothness can be observed (cf. Figure 2). Matérn kernels incorporate a smoothness parameter  $\nu > 0$  that permits greater flexibility in modeling such functions [31]. The following is a variation of the Matérn kernel for  $\nu = 1/2$ :

$$k_{\nu=1/2}(\mathbf{x}_i, \mathbf{x}_j) = \theta_0^2 \exp(-r), \quad (11)$$

where  $r^2(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i - \mathbf{x}_j)^\top \mathbf{\Lambda}(\mathbf{x}_i - \mathbf{x}_j)$  for some positive semidefinite matrix  $\mathbf{\Lambda}$ . For categorical variables, we implemented the following [12]:

$$k_{\theta}(\mathbf{x}_i, \mathbf{x}_j) = \exp(\sum_{\ell=1}^d (-\theta_{\ell} \delta(\mathbf{x}_i \neq \mathbf{x}_j))), \quad (12)$$

where  $d$  is the number of dimensions (*i.e.*, the number of configuration parameters),  $\theta_{\ell}$  adjust the scales along the function dimensions and  $\delta$  is a function gives the distance between two categorical variables using Kronecker delta [12], [25]. TL4CO uses different scales  $\{\theta_{\ell}, \ell = 1 \dots d\}$  on different dimensions as suggested in [31], [25], this technique is called Automatic Relevance Determination (ARD). After learning the hyper-parameters (*step 6*), if the  $\ell$ -th dimension turns out to be irrelevant, then  $\theta_{\ell}$  will be a small value, and therefore, will be discarded. This is particularly helpful in high dimensional spaces, where it is difficult to find the optimal configuration.

2) *Prior mean function*: While the kernel controls the structure of the estimated function, the prior mean  $\mu(\mathbf{x}) : \mathbb{X} \rightarrow \mathbb{R}$  provides a possible offset for our estimation. By default, this function is set to a constant  $\mu(\mathbf{x}) := \mu$ , which is inferred from the observations [25]. However, the prior mean function is a way of incorporating the expert knowledge, if it is available, then we can use this knowledge. Fortunately, we have collected extensive experimental measurements and based on our datasets (cf. Table I), we observed that typically, for Big Data systems, there is a significant distance between the minimum and the maximum of each function (cf. Figure 2). Therefore, a linear mean function  $\mu(\mathbf{x}) := \mathbf{a}\mathbf{x} + b$ , allows for more flexible structures, and provides a better fit for the data than a constant mean. We only need to learn the slope for each dimension and an offset (denoted by  $\mu_{\ell} = (\mathbf{a}, b)$ ).

3) *Learning parameters: marginal likelihood*: This section describe the *step 7* in Algorithm 1. Due to the heavy computation of the learning, this process is computed only every  $N_l$  iterations. For learning the hyper-parameters of the kernel and also the prior mean functions (cf. Sections III-E1 and III-E2), we maximize the marginal likelihood [25] of the observations  $\mathbb{S}_{1:t}$ . To do that, we train GP model (7) with  $\mathbb{S}_{1:t}$ . We optimize the marginal likelihood using multi-started quasi-Newton hill-climbers [23]. For this purpose, we use the off-the-shelf gpml library presented in [23]. Using the kernel defined in (12), we learn  $\theta := (\theta_{0:d}, \mu_{0:d}, \sigma^2)$  that comprises the hyper-parameters of the kernel and mean functions. The learning is performed iteratively resulting in a sequence of  $\theta_i$  for  $i = 1 \dots \lfloor \frac{N_{max}}{N_{\ell}} \rfloor$ .

4) *Observation noise*: The primary way for determining the noise variance  $\sigma$  in BO4CO is to use historical data: In Section II-B4, we have shown that such noise can be measured with a high confidence and the signal-to-noise ratios shows that such noise is stationary. The secondary alternative is to learn the noise variance sequentially as we collect new data. We treat them just as any other hyper-parameters, see Section III-E3.

## IV. EXPERIMENTAL RESULTS

### A. Implementation

From an implementation perspective, BO4CO consists of three major components: (i) an *optimization component* (cf. left part of Figure 6), (ii) an *experimental suite* (cf. right part of Figure 6) integrated via a (iii) *data broker*. The optimization component implements the model (re-)fitting (7) and criteria optimization (9) steps in Algorithm 1 and is developed in Matlab 2015b. The experimental suite component implements the facilities for automated deployment of topologies, performance measurements and data preparation and is developed in Java.

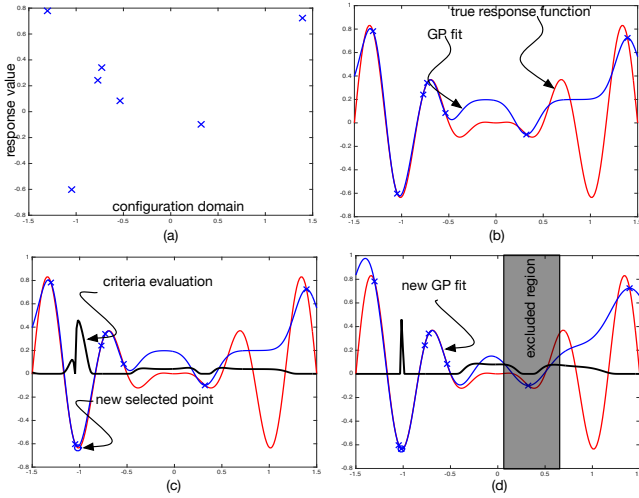


Fig. 8: Illustration of configuration parameter optimization: (a) initial observations; (b) a GP model fit; (c) choosing the next point; (d) refitting a new GP model.

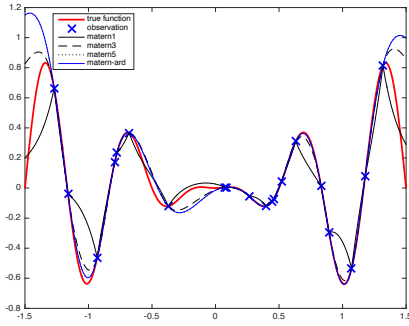


Fig. 9: The effect of changing the kernel in estimations.

The optimization component retrieves the initial design performance data and determines which configuration to try next using the procedure explained in III-C. The suite then deploys the *topology under test* on a testing cluster. The performance of the topology is then measured and the performance data will be used for model refitting. We have released the *code* and *data*: <https://github.com/dice-project/DICE-Configuration-BO4CO>.

In order to make BO4CO more practical and relevant for industrial use, we considered several implementation enhancements. In order to perform efficient GP model re-fitting, we implemented a covariance wrapper function that keeps the internal state for caching kernels and its derivatives, and can update kernel function by a single element. This was particularly helpful for learning the hyper-parameters at runtime.

## B. Experimental design

1) *Topologies under test and benchmark functions*: In this section, we evaluate BO4CO using 3 different Storm benchmarks: (i) WordCount, (ii) RollingSort, (iii) SOL. RollingSort implements a common pattern in real-time data analysis that performs rolling counts of incoming messages. RollingSort is used by Twitter for identifying trending topics. SOL is a *network intensive* topology, where the incoming messages will be routed through an inter-worker network.

Wordcount and RollingSort are standard benchmarks and are widely used in the community, *e.g.*, research papers [5] and industry scale benchmarks [11]. We have conducted all the experiments on 5 cloud clusters and with different sets of parameters resulted in datasets in Table I (see appendix [1]).

We also evaluate BO4CO with a number of benchmark functions, where we perform a synthetic experiment inside MATLAB in which a measurement is just a function evaluation: Branin(2D), Dixon-Szegö(2D), Hartmann(3D) and Rosenbrock(5D). These benchmark functions are commonly used in global optimization and configuration approaches [32], [28]. We particularly selected these because: (i) they have different curvature and (ii) they have multiple global minimizers, and (iii) they are of different dimensions.

2) *Baseline approaches*: The performance of BO4CO is compared with the 5 outstanding state-of-the-art approaches for configuration optimizations: SA [6], GA [2], HILL [32], PS [28] and Drift [27]. They are of different nature and use different search algorithms: simulated annealing, genetic algorithm, hill climbing, pattern search and adaptive search.

3) *Experimental considerations*: The performance statistics regarding each specific configuration has been collected over a window of 5 minutes (excluding the first two minutes of burn-in and the last minute of cluster cleaning). The first two minutes are excluded because the monitoring data are not stationary, while the last minute is the time given to the topology to fully process all messages. We then shut down the topology, clean the cluster and move on to the next experiment. We also replicated each runs of algorithms for 30 times in order to report the comparison results. Therefore, all the results presented in this paper are the mean performance over 30 runs.

4) *Cluster configuration*: We conducted all the experiments on 5 different multi-node clusters on three various cloud platforms, see [1] for more details. The reason behind this decision was twofold: (i) saving time in collecting experimental data by running topologies in parallel as some of the experiments supposed to run for several weeks, see Section II-B3. (ii) replicating the experiment with different processing node.

## C. Experimental analysis

In the following, we evaluate the performance of each approach as a function of the number of evaluations. So for each case, we report performance using the absolute distance of the *minimum function value* from the global minimum. Since we have measured all combinations of parameters in our datasets, we can measure this distance at each iteration.

1) *Benchmark functions global optimization*: The results for Branin in Figure 10(a) show that BO4CO outperforms the other approaches with three orders of magnitude, while this gap is only an order of magnitude for Dixon as in Figure 10(b). This difference can be associated to the fact that Dixon surface is more rugged than Branin [30]. For Branin, BO4CO finds the global minimum within the first 40 iterations, while even SA stalls on a local minimum. The rest, including GA, HILL, PS perform similarly to each other throughout the experiment and reach a local minimum, which BO4CO finds only within the first 10 iterations. For Dixon, BO4CO gets close to the global minimum within the first 20 iterations, while the best performers (*i.e.*, PS and HILL) approach to points an order of

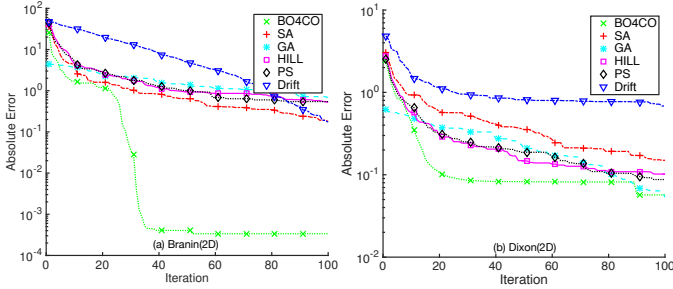


Fig. 10: Branin(2D), Dixon(2D) test function optimization.

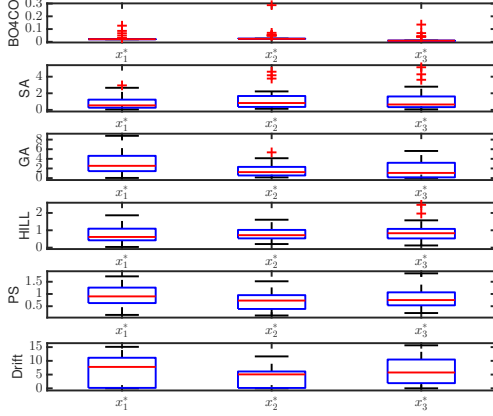


Fig. 11: Distance to the three Branin's minimizers.

magnitude away comparing with BO4CO. Branin function has 3 global minimizers at  $x_1^* = (\pi, 12.27)$ ,  $x_2^* = (\pi, 2.27)$ ,  $x_3^* = (9.42, 2.47)$ , interestingly comparing to baselines, BO4CO gets close to all minimizers (cf. Figure 11). Therefore, BO4CO gains information on all minimizers as opposed to baselines.

The results for Hartmann in Figure 12(a) show that BO4CO decreases the absolute error quickly after 20 iterations, but only approaches to the global minimum after 120 iterations. Neither of the baseline approaches get close to the global minimum even after 150 iterations. The good performance of BO4CO is also confirmed in the case of Rosenbrock, as shown in Figure 12(b). BO4CO finds the optimum in such large space only after 60 iterations, while GA, HILL, PS and Drift perform poorly with an error of three orders of magnitude higher than our approach. However, SA performs well with an order of magnitude away from the ones found by BO4CO.

2) *Storm configuration optimization*: We now discuss the results of BO4CO on the Storm datasets in Table I: SOL(6D), RollingSort(6D), WordCount(3D,5D).

The results for SOL in Figure 13(a) show that BO4CO decreases the optimality gap within the first 10 iterations and decreases this gap until iteration 200 and does not get trapped into a local minimum. Instead, baseline approaches like Drift and GA get trapped into a local minimum in early iterations, while HILL and PS get stuck some iterations later at 120. Among the baselines, SA performs the best and it decreases the optimality gap in the first 70 iterations, however, it gets stuck to a local optimum thereafter.

The results for RollingSort in Figure 13(b) are similar to the SOL ones. BO4CO decreases the error considerably in

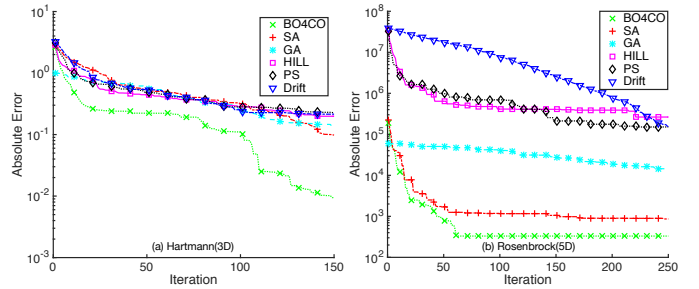


Fig. 12: Hartmann(3D), Rosenbrock(5D) optimization.

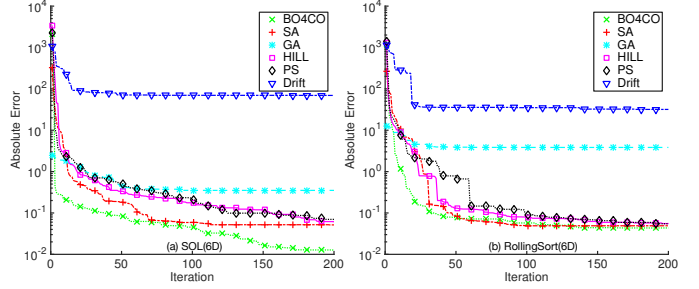


Fig. 13: SOL(6D), RollingSort(6D) optimization.

the first 50 iterations, while the baseline approaches, except SA and HILL, perform poorly in that period. However, during iterations 100-200, the rests, except GA and Drift, find configuration with close performance as the ones BO4CO finds.

For WordCount (Figure 14(a,b)) the results are different. BO4CO outperforms the best baseline performer, *i.e.*, SA, by an order of magnitude, while the others by at least two orders of magnitude. Among the baselines, SA performs the best for WordCount(3D), while for WordCount(5D) dataset, HILL and PS performs better in the first 50 iterations.

Summarizing, while the results for the Storm benchmarks are consistent with the ones we observed for the benchmark functions, it shows a clear gain in favor of BO4CO, with at least an order of magnitude in the initial iterations. In each case, BO4CO finds a better configuration much more quickly than baselines. As opposed to the benchmark functions, SA consistently outperforms the rest of baseline approaches. To highlight this achievement, note that 50 iterations for a dataset like SOL (6D) is only 1% of the total number of possible tests for finding the optimum configurations and identifying such configurations with a latency close to the global optimum can save a considerable time and cost.

#### D. Sensitivity analysis

1) *Prediction accuracy of the learned GP model*: Since BO4CO does not uniformly sample the configuration space (cf. Figure 5.8), we speculated that the GP models trained in BO4CO are not useful for predicting the performance of configurations that have not been experimented. However, when we compared the GP model on the WordCount, it was much more accurate than the ones of polynomial regressions (see Figure 15). This shows a clear advantage over design of experiments (DoE), which normally uses first-order and second-order polynomials. The root mean squared error

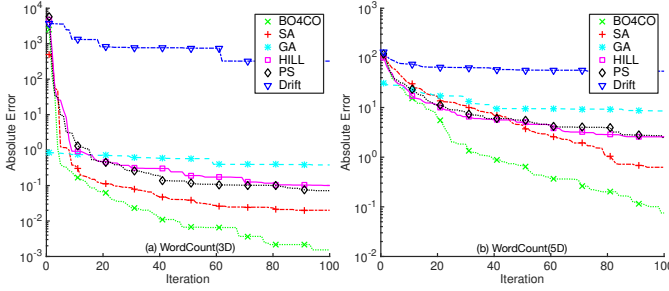


Fig. 14: WordCount(3D,5D) configuration optimization.

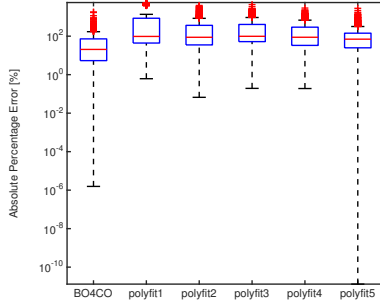


Fig. 15: Absolute percentage error of predictions made by BO4CO's GP fit after 100 iterations vs multivariate polynomial regression models for WordCount(3D) dataset.

(RMSE) for Branin and Dixon in 16(a) clearly show that the GP models can provide accurate predictions after 20 iterations. This fast learning rate can be associated to the power of GPs for regressions [31]. We further compared the prediction accuracy of the GP models trained in BO4CO with several machine learning models (including M5Tree, Regression Tree, LWP, PRIM [31]) in Figure 16(b) and we observed that the GP model predictions were more accurate, while the accuracy of other models either did not improve (*e.g.*, M5Tree) or was deteriorated (*e.g.*, PRIM, polyfit5) due to over-fitting.

2) *Exploitation vs. exploration*: In (10),  $\kappa$  adjusts the exploitation-exploration: small  $\kappa$  means high exploitation, while a large  $\kappa$  means a high exploration. The results in Figure 17(a) show that using a relatively high exploration (*i.e.*,  $\kappa = 8$ ) performs better up to an order of magnitude comparing with high exploitations (*i.e.*,  $\kappa = 0.1$ ). However, for  $\kappa = 0.1, 1$  exploiting the mean estimates improves the performance at early iterations comparing with higher explorations cases as in  $\kappa = 6$ . This observation motivated us to tune  $\kappa$  dynamically by using a lower value at early iterations to exploit the knowledge gained through the initial design and set a higher value later on, see Section III-C. The result for WordCount in Figure 17(b) confirms that adaptive  $\kappa$  improves the performance considerably over constant  $\kappa$ . Figure 18 shows that when we increase  $\kappa$  with a higher rate (*cf.* Figure 7), it will improve the performance. However, the results in Figure 17(a) suggest using a high value of exploration, this should not be set to an extreme where this makes the mean estimates ineffective. As in Figure 17(a), it performs 4 orders of magnitude worse when we ignore the mean estimates (*i.e.*,  $\mu_t := 0$ ).

3) *Bootstrapping vs no bootstrapping*: BO4CO uses 1hd design in order to bootstrap the search process. The results for

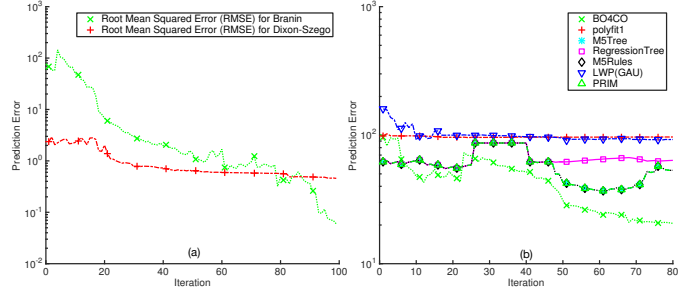


Fig. 16: (a) improving accuracy of GP models over time, (b) comparing prediction accuracy of GPs with other machine learning models on wc(6D).

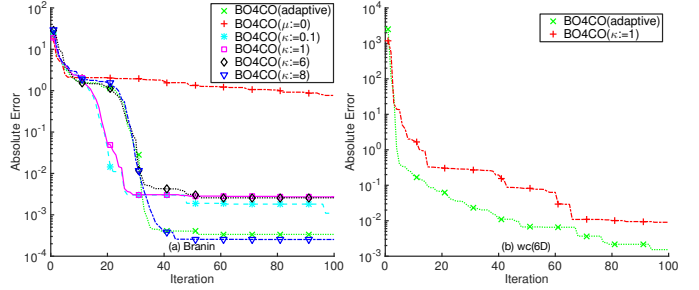


Fig. 17: Exploitation vs exploration (a) Branin, (b) wc(6D).

Hartmann and WordCount in Figure 19(a,b) confirms that this choice provides a good opportunity in order to explore along all dimensions and not to trap into local optimum thereafter. However, the results in Figure 19(b) suggest that a high number of initial design may deteriorate the goal of finding the optimum early.

## V. DISCUSSIONS

### A. Computational and memory requirements

The exact inference BO4CO uses for fitting a GP model to the  $t$  observed data is  $O(t^3)$  because of inversion of kernel  $K^{-1}$  in (7). We could in principle compute the Cholesky decomposition and use it for subsequent predictions, which would lower the complexity to  $O(t^2)$ . However, since in BO4CO we learn the kernel hyper-parameters every  $N_\ell$  iterations, Cholesky decomposition must be re-computed, therefore the complexity is in principle  $O(t^2 \times t/N_\ell)$ , where the additional factor of  $t/N_\ell$  counts the expected number of iterations. Figure 20 provides the computation time for finding the next configuration in Algorithm 1 including (i) the likelihood maximization (ii) the GP model fitting in (7), and (iii) the LCB calculation in (10) altogether for 5 datasets in Table I. The time is measured running BO4CO on a MacBook Pro with 2.5 GHz Intel Core i7 CPU and 16GB of Memory. The computation time in larger datasets (RollingSort(6D), SOL(6D), WordCount(6D)) is higher than those with less data and lower dimensions (WordCount(3,5D)). Moreover, the computation time increases over time since the matrix size for Cholesky inversion gets larger.

BO4CO requires to store 3 vectors of size  $|\mathbb{X}|$  for mean, variance and LCB estimates and a matrix of size  $|\mathbb{S}_{1:t}| \times |\mathbb{S}_{1:t}|$



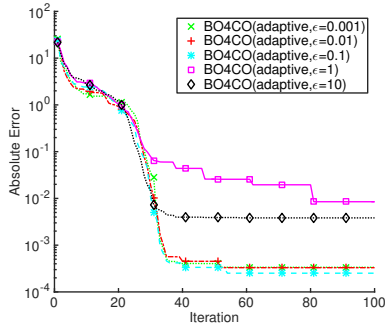


Fig. 18: Exploitation vs. exploration: compare the rate of changes in  $\kappa$  with different value of  $\epsilon$  in Figure 7.

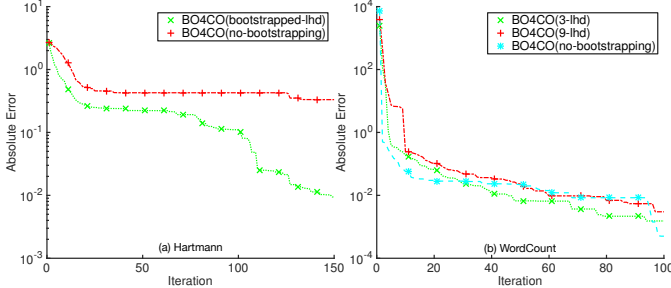


Fig. 19: Bootstrapping vs no bootstrapping: acceleration of performance due to bootstrapping.

for  $K$  and of size  $|\mathbb{S}_{1:t}|$  for observations, making the memory requirement of  $O(3|\mathbb{X}| + |\mathbb{S}_{1:N_{max}}|(|\mathbb{S}_{1:N_{max}}| + 1))$  in total.

### B. BO4CO in practice

*Feasibility.* We have invested more than three months (24/7) for measurements of the systems in Table I. However, our approach would require much less measurements (results in Section IV-C), as majority of measurements were meant to evaluate our approach. Since we needed to report the distance to the *global minimum*, we had to measure all combinations of parameters (*i.e.*, exhaustive search). This leads to an extensive number of measurements only for evaluation purposes.

*Extensibility.* We have integrated BO4CO with Continuous Integration, delivery, deployment and monitoring tools in a DevOps pipeline as a part of H2020 DICE project, see [1]. BO4CO acts as the configuration tuning tool for Big Data systems in this ecosystem. We also provided BO4CO functionalities via RESTfull APIs, therefore it can be easily integrated in any ecosystem that require configuration tuning.

*Usability.* BO4CO is easy to use, end users only need to determine the parameters of interests as well as experimental parameters and then the tool automatically sets the optimized parameters. Currently, BO4CO supports Apache Storm and Cassandra. However, it is designed to be extensible.

*Scalability.* The scalability bottleneck is experimentation. The running time of the cubic algorithm is of the order of milliseconds (cf. Figure 20). Each of these experiments takes more than 10 minutes, orders of magnitude over BO4CO.

### C. Threats to validity

*Internal Validity.* We have repeated all measurements 30 times and have used averages to compare the performance. We

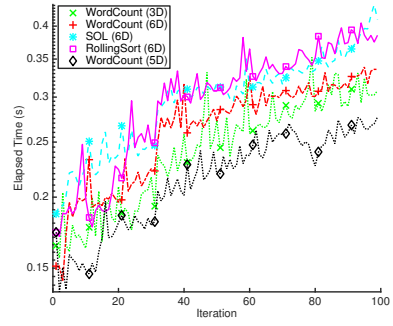


Fig. 20: Runtime overhead of BO4CO (excluding the experiment time) is in the scale of few hundred milliseconds.

have also used different cloud platforms for our experiments (see supplementary materials in [1]).

*External Validity.* The benchmark systems have different architectures, varying numbers of parameters and are from different domains. Some systems (*e.g.*, WC) retrieve configuration at compile time, others (*e.g.*, RS, SOL) at load time. BO4CO works for any configurable systems that have parameters of type continuous, discrete and categorical.

## VI. RELATED WORK

There exist several categories of approaches to address the system configuration problem, as listed in Table II.

*Rule-based:* In this category, domain experts create a repository of rules that is able to recommend a good configuration. *e.g.*, IBM DB2 Configuration Advisor [16]. The Advisor asks administrators a series of questions, *e.g.*, does the workload is CPU or memory intensive? Based on the answers, it recommends a configuration. However, for multi-dimensional spaces such as SPS in which the configuration parameters have unknown non-linear relationship, this approach is naive [28].

*Design of experiments:* DoE conducts exhaustive experiments for different combinations of parameters in order to find influential factors [7]. Although DoE is regarded as a classical approach for application configuration, in multi-dimensional spaces performing naive experiments without any sequential feedback from real environment is infeasible and costly.

*Model-based:* This category conducts a series of experiments where each runs the system using a chosen configuration to observe its performance. Each experiment produces a  $(x, f(x))$  sample. A (statistical) model can then be trained from these samples and used to find good configurations. However, an exhaustive set of experiments, usually above the limited budget, need to be conducted to provide a representative data sets, otherwise the prediction based on the trained model will not be reliable (cf. Figure 15). White box [19] and black box models [14], [34], [26] have been proposed.

*Search-based:* In this approach, also known as *sequential design*, experiments can be performed sequentially where the next set of experiments is determined based on an analysis of the previous data. In each iteration a (statistical) model is fitted to the data and this model guides the selection of the next configuration. Evolutionary search algorithms such as simulated annealing, recursive random search [33], genetic algorithm [2], hill climbing [32], sampling [26] and Covariance Matrix Adaptation [24] have been adopted.

TABLE II: Systems configuration (auto-tuning) approaches.

| Category            | Empirical | Black box | Interactions | Approaches       |
|---------------------|-----------|-----------|--------------|------------------|
| Rule-based          | No        | No        | No           | [16]             |
| DoE                 | Yes       | Yes       | Yes          | [29]             |
| Model (white-box)   | Partially | No        | No           | [19]             |
| Model (blackbox)    | Yes       | Yes       | Yes          | [14], [34], [26] |
| Search (sequential) | Yes       | Yes       | Yes          | [32], [22], [6]  |
| Search (evol.)      | Partially | Yes       | Yes          | [8], [2], [33]   |
| Space reduction     | Yes       | Yes       | Yes          | [35]             |
| Online learning     | Yes       | Yes       | No           | [3]              |
| Knowledge transfer  | Yes       | Yes       | Yes          | [4]              |

Empirical column describes whether the configuration is based on real data. Interactions describes whether the non-linear interactions can be supported.

*Learning-based:* There exists some approaches that employ offline and online learning (e.g., reinforcement learning) to enable online system configuration adaptation [3]. The approaches in this category, as opposed to the other approaches, try to find optimum configurations and adapt it when the situations has been changed at runtime. However, the main shortcoming is the learning that may converge very slowly [13]. The learning time can be shortened if the online learning entangled with offline training [3]. This can be even further improved if we discover the relationship between parameters (e.g., [35], [4]) and exploit such knowledge at runtime.

*Knowledge transfer:* There exist some approaches that reduce the configuration space by exploiting some knowledge about configuration parameters. Approaches like [4] use the dependence between the parameters in one system to facilitate finding optimal configuration in other systems. They embed the experience in a well-defined structure like Bayesian network through which the generation of new experiments can be guided toward the optimal region in other systems.

*Concluding remarks:* BO4CO is model-based that is entangled with a search mechanism exploring the unknown zones using a uncertainty-aware machine learning model. The exploration becomes possible due to the confidence interval estimates. Therefore, BO4CO has a commonality with all above categories except the rule-based.

## VII. CONCLUSIONS

This paper proposes BO4CO, an approach for locating optimal configurations using ideas of carefully choosing where to sample by sequentially reducing uncertainty in the response surface approximation in order to reduce the number of measurements. BO4CO sequentially gains knowledge about the posterior distribution of the minimizers. We experimentally demonstrate that BO4CO is able to locate the minimum of some benchmark functions as well as optimal configurations within real stream datasets accurately compared to five baseline approaches. We have carried out extensive experiments with three different stream benchmarks running on Apache Storm. The experimental results demonstrate that BO4CO outperforms the baselines in terms of distance to the optimum performance with at least an order of magnitude. We have also provided some evidence that the learned model throughout the search process can be also useful for performance predictions.

## REFERENCES

- [1] appendix: [www.doc.ic.ac.uk/~pjamshid/PDF/appendix-mascots16.pdf](http://www.doc.ic.ac.uk/~pjamshid/PDF/appendix-mascots16.pdf).
- [2] B. Behzad et al. Taming parallel I/O complexity with auto-tuning. In *Proc. ACM SC*, 2013.

- [3] X. Bu, J. Rao, and C.-Z. Xu. A reinforcement learning approach to online Web systems auto-configuration. In *Proc. ICDCS*, 2009.
- [4] H. Chen et al. Experience transfer for the configuration tuning in large scale computing systems. *SIGMETRICS*, 37(2):51–52, 2009.
- [5] A. Ghazal, T. Rabl, M. Hu, F. Raab, M. Poess, A. Crolotte, and H.-A. Jacobsen. Bigbench: towards an industry standard benchmark for big data analytics. In *SIGMOD*, pages 1197–1208. ACM, 2013.
- [6] J. Guo et al. Evaluating the role of optimization-specific search heuristics in effective autotuning. Technical report, 2010.
- [7] M. Hall. Correlation-based feature selection for machine learning. *Thesis, Waikato*, 1999.
- [8] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation*, 9(2):159–195, 2001.
- [9] C. Henard, M. Papadakis, M. Harman, and Y. Le Traon. Combining multi-objective search and constraint solving for configuring large software product lines. In *ICSE*, pages 517–528. IEEE, 2015.
- [10] M. Hirzel et al. A catalog of stream processing optimizations. *ACM Computing Surveys*, 46(4):46, 2014.
- [11] S. Huang, J. Huang, Y. Liu, L. Yi, and J. Dai. Hibench: A representative and comprehensive hadoop benchmark suite. In *Proc. ICDE*, 2010.
- [12] F. Hutter. Automated configuration of algorithms for solving hard computational problems. *Thesis*, 2009.
- [13] P. Jamshidi et al. Self-learning cloud controllers: Fuzzy Q-learning for knowledge evolution. In *Proc. ICCAC*, 2015.
- [14] T. Johnston et al. Performance tuning of mapreduce jobs using surrogate-based modeling. *ICCS*, 2015.
- [15] J. P. C. Kleijnen. Response surface methodology. *Wiley Interdisciplinary Reviews*, 2010.
- [16] E. Kwan et al. Automatic database configuration for DB2 universal database: Compressing years of performance expertise into seconds of execution. In *Proc. BTW*, volume 20, 2003.
- [17] J. Liebig, A. von Rhein, C. Kästner, S. Apel, J. Dörre, and C. Lengauer. Scalable analysis of variable software. In *FSE*. ACM, 2013.
- [18] D. J. Lizotte et al. An experimental methodology for response surface optimization methods. *Journal of Global Optimization*, 53:699–736, 2012.
- [19] D. A. Menascé et al. Preserving QoS of e-commerce sites through self-tuning: a performance model approach. In *Proc. EC*, pages 224–234. ACM, 2001.
- [20] D. C. Montgomery. *Design and analysis of experiments*. John Wiley & Sons, 2008.
- [21] Z. Nabi, E. Bouillet, A. Bainbridge, and C. Thomas. Of streams and storms. *IBM White Paper*, 2014.
- [22] T. Osogami and S. Kato. Optimizing system configurations quickly by guessing at the performance. In *Proc. SIGMETRICS*, volume 35, 2007.
- [23] C. E. Rasmussen and H. Nickisch. Gaussian processes for machine learning (gpml) toolbox. *The Journal of Machine Learning Research*, 11, 2010.
- [24] A. Saboori et al. Autotuning configurations in distributed systems for performance improvements using evolutionary strategies. In *Proc. ICDCS*, 2008.
- [25] B. Shahriari et al. Taking the human out of the loop: a review of bayesian optimization. Technical report, 2015.
- [26] N. Siegmund et al. Performance-influence models for highly configurable systems. In *Proc. FSE*, 2015.
- [27] J. Sun et al. Random drift particle swarm optimization. *arXiv preprint arXiv:1306.2863*, 2013.
- [28] R. Thonangi et al. Finding good configurations in high-dimensional spaces: Doing more with less. In *Proc. MASCOTS*. IEEE, 2008.
- [29] T. Ustinova and P. Jamshidi. Modelling multi-tier enterprise applications behaviour with design of experiments technique. In *Proc. QUDOS*. ACM, 2015.
- [30] T. Weise. Global optimization algorithms-theory and application. *Self-Published*, 2009.
- [31] C. K. Williams and C. E. Rasmussen. Gaussian processes for machine learning. *MIT Press*, 2006.
- [32] B. Xi et al. A smart hill-climbing algorithm for application server configuration. In *WWW*, 2004.
- [33] T. Ye and S. Kalyanaraman. A recursive random search algorithm for large-scale network parameter configuration. *SIGMETRICS*, 31(1):196–205, 2003.
- [34] N. Yigitbasi et al. Towards machine learning-based auto-tuning of mapreduce. In *Proc. MASCOTS*, 2013.
- [35] W. Zheng, R. Bianchini, and T. D. Nguyen. Automatic configuration of internet services. *ACM SIGOPS Operating Systems Review*, 41(3):219, 2007.